# System Verification and Validation Plan for Software Engineering

Team 22, Ghoti
Hemal Rajput
Gurmanjot Minhas
James Araiche
Lily Porter
Sarah Dorfman

November 3, 2026

# Revision History

| Date | Version | Notes |
|---|---|---|
| Date 1 | 1.0 | Notes |
| Date 2 | 1.1 | Notes |

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

# Contents


**1 Symbols, Abbreviations, and Acronyms** **iv**

**2 General Information** **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

**3 Plan** **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 2
- 3.3 Design Verification . . . 2
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 3
- 3.6 Automated Testing and Verification Tools . . . 3
- 3.7 Software Validation . . . 4

**4 System Tests** **4**
- 4.1 Tests for Functional Requirements . . . 4
  - 4.1.1 Area of Testing1 . . . 4
  - 4.1.2 Area of Testing2 . . . 5
- 4.2 Tests for Nonfunctional Requirements . . . 5
  - 4.2.1 Accuracy and responsiveness . . . 5
  - 4.2.2 Usability and patient protection . . . 7
  - 4.2.3 Maintainability and portability . . . 9
  - 4.2.4 Reliability, security and release review . . . 10
- 4.3 Legal and Licensing . . . 12
  - 4.3.1 Area of Testing1 . . . 12
  - 4.3.2 Area of Testing2 . . . 13
- 4.4 Traceability Between Test Cases and Requirements . . . 13

**5 Unit Test Description** **13**
- 5.1 Unit Testing Scope . . . 13
- 5.2 Tests for Functional Requirements . . . 14
  - 5.2.1 Module 1 . . . 14
  - 5.2.2 Module 2 . . . 15

5.3 Tests for Nonfunctional Requirements . . . 15
5.3.1 Module ? . . . 15
5.3.2 Module ? . . . 16
5.4 Traceability Between Test Cases and Modules . . . 16

**6 Appendix** **17**
6.1 Symbolic Parameters . . . 17
6.2 Usability Survey Questions? . . . 17

**7 Appendix — Generative AI Use Disclosure** **18**
7.1 AI Tools Used . . . 18
7.2 How and Where Used . . . 18
7.3 Verification of AI-Generated Content . . . 18

**8 Appendix — Reflection** **19**

# List of Tables

[Remove this section if it isn't needed —SS]

# List of Figures

[Remove this section if it isn't needed —SS]

# 1 Symbols, Abbreviations, and Acronyms

| symbol | description |
|---|---|
| T | Test |

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

# 2 General Information

## 2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

## 2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

## 2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

### 2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

## 3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

### 3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

### 3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

### 3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

## 3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

## 3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

## 3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

### 3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

## 4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

### 4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

#### 4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

**Title for Test**

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

#### 4.1.2 Area of Testing2

...

### 4.2 Tests for Nonfunctional Requirements

#### 4.2.1 Accuracy and responsiveness

Use a consistent evaluation set so measurements can be compared across model changes, as required by the Development Plan.

**Measure phoneme detection accuracy**

1. **ST-N01**

   Type: Dynamic, Automatic

   Initial State: A set of labelled children's speech clips from TalkBank (e.g. PhonBank), with reference phoneme transcriptions, is loaded. Clips used for tuning are excluded.

   Input/Condition: Run every clip through the recognizer and compare the output phonemes against the labelled reference phonemes.

   Output/Result: Phoneme identification accuracy is at least 87% (NFR-ACCU1; FR-SA1, FR-SA2, FR-SA3). Report accuracy overall and per target phoneme.

   How test will be performed: Align recognized and reference phonemes for each clip. Accuracy is the number of correctly identified phonemes divided by the total number of reference phonemes. Rerun the same clip set after each model change.

**Measure processing time from audio capture to phoneme estimation**

1. **ST-N02**

   Type: Dynamic, Automatic measurement

   Initial State: The device and software are configured to record speech and process it.

   Input/Condition: Input audio file for a particular word.

   Output/Result: For each case, phonemic estimation results are visible in less than 10 seconds (NFR-PER1). Record duration for each processing completion and categorize by the specific phrase. Report all observations plus processing time divided by audio duration, median, and maximum.

   How test will be performed: Measure from completed audio capture/submission to rendered result, including transfer and rendering, with at least 30 runs per representative condition as a planned sampling choice. Log

hardware, network, clip duration and model size. Resolve whether capture time belongs in the requirement's timing boundary before acceptance. Basis: NFR-PER1 sets a strict less than 10 seconds criterion; Development Plan Section 10.5 adds real-time factor and end to end latency.

### 4.2.2 Usability and patient protection

Use SLP task observations for navigation and efficiency. Assess the child-facing boundary separately from clinician access.

**Learn the workflow within one hour**

1. **ST-N03**

   Type: Dynamic, Manual

   Initial State: An SLP unfamiliar with the application has standard onboarding material and synthetic data.

   Input/Condition: Within a one hour session, ask the SLP to configure practice, record audio and analyze it, override a result, inspect charts and observe patient history.

   Output/Result: The SLP completes the core navigation tasks independently within one hour of use (NFR-USA1). Record task times, assistance, failures and comments.

   How test will be performed: The UI/UX Lead observes without guiding the final task run. They record mistakes, time to complete each task, and any questions or comments. The SLP is asked to provide feedback on the experience.

1. **ST-N04**

   Type: Dynamic usability and recovery checks

   Initial State: A test SLP account exists with synthetic critical data (for example, patient records and session recordings).

   Input/Condition: As the SLP, attempt to delete each type of critical data item. Where a confirmation prompt appears, test both cancelling

and confirming. Where deletion is allowed without a prompt, attempt recovery at several points within 72 hours and again after 72 hours have elapsed.

Output/Result: No critical data item is permanently deleted without either a confirmation step or a recovery option (NFR-USA2). Cancelling a confirmation leaves the data unchanged. Deleted data is fully restored when recovery is requested within 72 hours, with contents identical to the original.

How test will be performed: The Development Lead and an independent reviewer walk through each data type, automated script compares restored contents against the original synthetic data, and check the system clock or retention settings to verify the 72-hour window. Record any data type that can be lost without a safeguard; review is blocked if the critical data list is absent. Basis: NFR-USA2.

**Compare workflow time with manual tools**

1. **ST-N05**
   Type: Dynamic, Manual comparative measurement

   Initial State: An SLP has learned the application and has equivalent paper or spreadsheet tools.

   Input/Condition: Complete matched session and after-session documentation tasks with identical attempts using software and manual tools.

   Output/Result: Total software task time is lower than manual task time, with equivalent completed records (NFR-USA3). Report paired times, corrections and assistance; a faster incomplete record does not pass.

   How test will be performed: The UI/UX Lead checks where multiple participants are available and records session and after-session times separately and together. Report sample size and learning effects. Basis: NFR-USA3 requires less time than equivalent manual tools, including work outside the session.

**Restrict the child-facing experience**

1. **ST-N06**

   Type: Dynamic, Mixed manual and automatic

   Initial State: Child practice mode is active; clinician functions require the configured access boundary.

   Input/Condition: Try visible controls, back navigation, direct clinician URLs and direct API requests from the child context.

   Output/Result: The child can use the gamified experience but cannot access patient records, analytics administration or clinician controls beyond that experience (NFR-USA4). Direct navigation and API calls do not bypass restrictions.

   How test will be performed: The Systems Architect/Development Lead tests each route with the UI/UX Lead and captures denied requests and screenshots. Verify returning to clinician mode requires the configured authorization. Basis: NFR-USA4 limits the child's access to the practice experience.

### 4.2.3 Maintainability and portability

Checks supported hardware and software environments. Check deployment, patching, and extension.

1. **ST-N07**

   Type: Dynamic performance check

   Initial State: A clean build environment and a prepared patch exist, and the approved build and deployment pipeline is available.

   Input/Condition: Apply a representative patch and run the pipeline from build start to a completed deployment, timing each stage.

   Output/Result: Building and deploying the patch completes successfully in less than 30 minutes (NFR-MAI3).

   How test will be performed: The Development Lead and an independent reviewer time the pipeline run from pipeline logs and repeat it three times. Record durations and any failures; review is blocked if the pipeline is unavailable. Basis: NFR-MAI3.

**Verify platform and browser compatibility**

1. **ST-N08**

   Type: Dynamic, Manual

   Initial State: A versioned compatibility matrix is recorded with physical devices for microphone checks.

   Input/Condition: Exercise Windows with Chrome, Firefox and Edge; macOS with Safari, Chrome and Firefox; iPad/iOS with Safari; and Android tablet with Chrome. Include desktop, laptop and tablet form factors.

   Output/Result: Setup, microphone permission and capture, analysis, SLP controls, IPA, charts and history operate without blocking layout or functional defects in every tested combination (NFR-POR1, NFR-POR2, NFR-POR3; FR-RDV9).

   How test will be performed: Repeat a compact ST-F01/F03/F08/F09/F11 workflow in each matrix entry; inspect orientation and touch controls on tablets; test microphone functionality. Add any other claimed supported combination to the matrix before declaring support. Basis: The SRS lists four operating systems, four browsers and three device types; the matrix is a representative test plan, not proof of every OS-browser combination.

### 4.2.4 Reliability, security and release review

These checks address operational hazards, including data loss, and unauthorized access.

**Verify operating periods and maintenance scheduling**

1. **ST-N09**

   Type: Mixed Dynamic and Static

   Initial State: Deployment operating timezone, full and minimal service definitions, load and maintenance policy are approved.

Input/Condition: Exercise login, storage, retrieval and analysis around 06:00 and 00:00; run an accelerated schedule check and inspect the maintenance calendar.

Output/Result: The approved full service level operates from 06:00 to 00:00 and the approved minimum remains available from 00:00 to 06:00. Routine maintenance is scheduled in minimal hours at the approved interpretation of the one to two month interval (NFR-REL2, NFR-REL3, NFR-REL4). Check server latency and ensure it is under 10 seconds for this period (NFR-REL1).

How test will be performed: Use a controlled clock and scheduler inspection, then planned observation under the approved workload.

**Protect data and secure communications**

1. **ST-N10**

Type: Static inspection and Dynamic security checks

Initial State: The approved PHI inventory and encryption baseline identify every component and storage location.

Input/Condition: Create and transfer synthetic PHI; inspect browser-to-API, service-to-service and database traffic, persistent stores, logs, temporary files and backups.

Output/Result: PHI is encrypted from collection through its approved storage lifecycle; intercomponent traffic uses the approved secure protocol (NFR-SAP1, NFR-SAP2; NFR-LAL1). No unintended plaintext PHI appears in inspected storage or traffic. Approved key access and certificate validation controls are active.

How test will be performed: The Systems Architect/Development Lead and an independent team reviewer inspect configuration, packet traces and synthetic storage samples. Test denial of an untrusted certificate and an insecure endpoint. Record coverage and gaps; review is blocked if the baseline is absent. Basis: NFR-SAP1 and NFR-SAP2 require encryption; exact cryptographic acceptance comes from an approved security baseline, not this plan.

## 4.3 Legal and Licensing

Checks to ensure that the system is PHIPA compliant and open source.

1. **ST-N11**

   Type: Static inspection and Dynamic security checks

   Initial State: The approved PHIPA checklist and deployment region list exist.

   Input/Condition: Process synthetic patient records through storage, transfer and access paths; attempt access with unauthorized roles; review the repository license and dependency manifest.

   Output/Result: Synthetic PHI is stored, transferred and accessed only as the PHIPA checklist permits, and stays in approved residency locations (NFR-LAL1). Unauthorized access is denied. The repository includes the GNU GPL text and all dependencies are GPL-compatible (NFR-LAL2).

   How test will be performed: The Development Lead and an independent reviewer compare configuration, storage locations and access logs against the checklist, and run a dependency license scan. Record findings; review is blocked if the checklist is absent. Basis: NFR-LAL1 and NFR-LAL2; legal sign-off is outside this plan.

### 4.3.1 Area of Testing1

**Title for Test**

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

#### 4.3.2 Area of Testing2

...

### 4.4 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

## 5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

### 5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on

verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

## 5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

### 5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

3. ...

#### 5.2.2 Module 2

...

### 5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

#### 5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

How test will be performed:

#### 5.3.2 Module ?

...

### 5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

## References

Author Author. System requirements specification. https://github.com/..., 2019.

# 6 Appendix

This is where you can place additional information.

## 6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

## 6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

# 7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

## 7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

## 7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

## 7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

# 8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?